Huffman Coding

- Huffman coding is a form of statistical coding
- Not all characters occur with the same frequency!
- Yet all characters are allocated the same amount of space
  -> 1 char = 1 byte, be it e or x
- Any savings in tailoring codes to frequency of character?
- Code word lengths are no longer fixed like ASCII.
- Code word lengths vary and will be shorter for the more frequently used characters.

Huffman Coding is a technique of compressing data to reduce its size without losing any details.

Problem: Not all characters occur with the same frequency. Yet all characters are allocated the same amount of space. According to ASCII, 1 character = 1 byte or 8 bits.

Solution: The idea is to assign variable-length codes to the characters, lengths of the assigned codes are based on the frequencies of corresponding characters. More frequently used characters have shorter codes. By reducing the length of codes for the characters that appear most often, the overall size of the encoded data is minimized.

***Simulation

Hello

H - 1

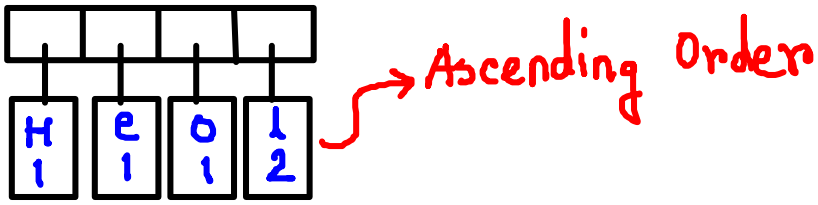
e - 1

l - 2

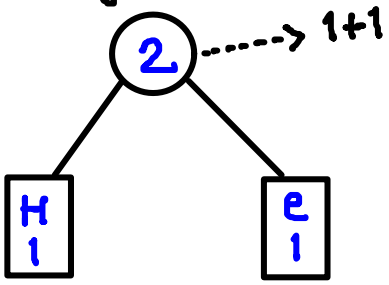
O - 1

Priority Queue :

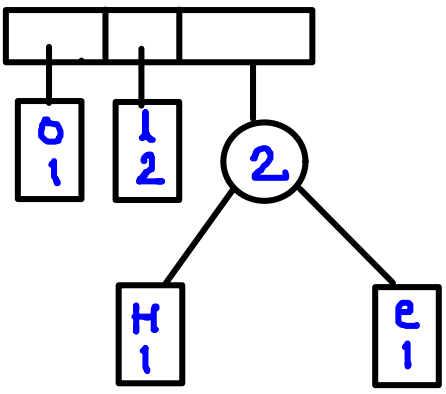
(1)



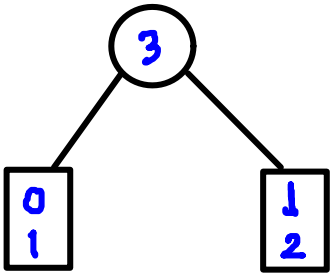
(2) Extracting H and e:



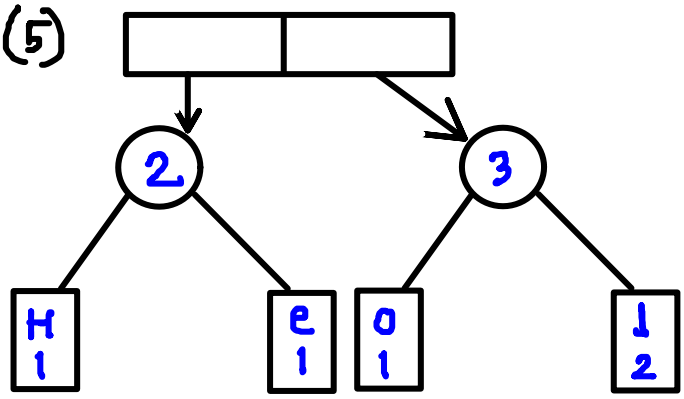
(3)



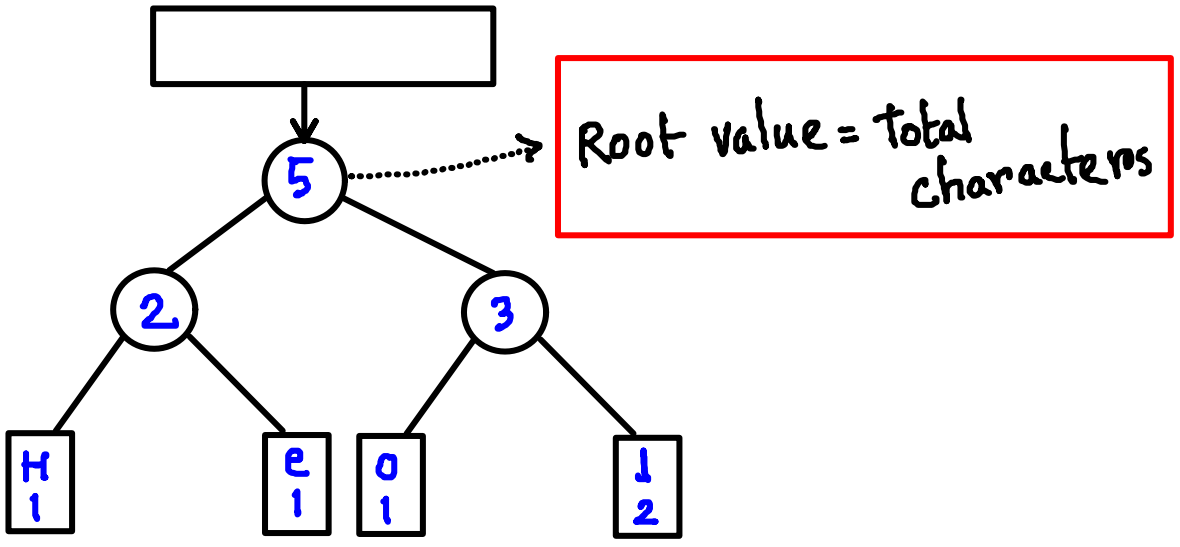
(4) Extracting 0 and 1 :



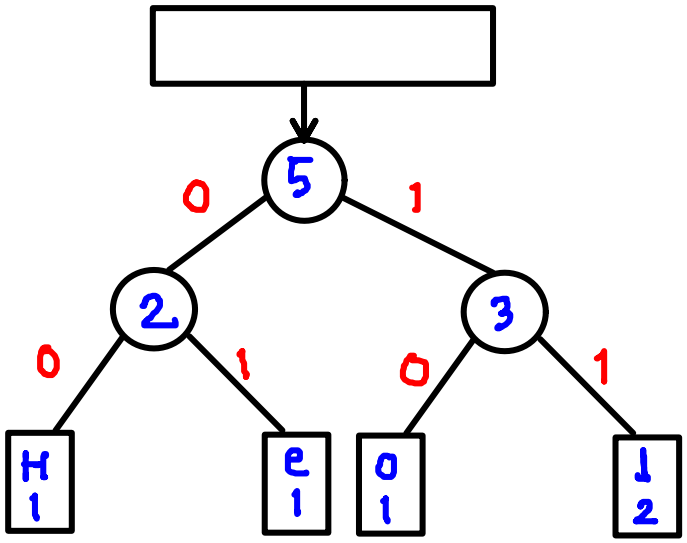
(5)



(6)



(7)



Left → 0
Right → 1

H → 00
e → 01
O → 10
l → 11

Total = 8 bits.
If it was done by ASCII,
total = 5x8 = 40 bits.

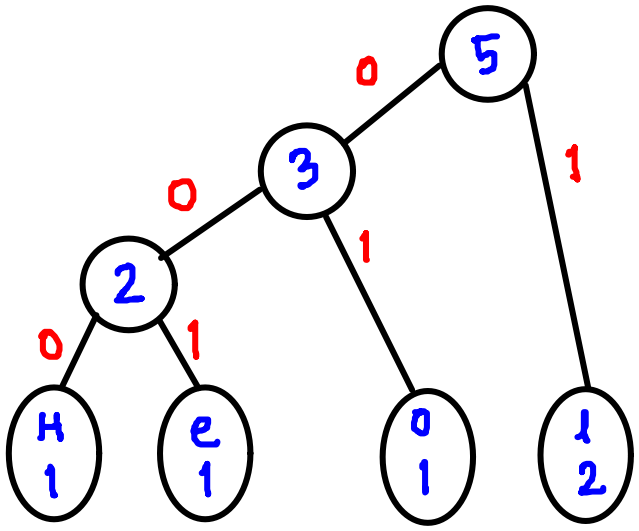
This is actually the Most Perfect way. No chancce of mistake while building trees. But this is also very time consuming.

There is a shortcut way which saves a lot of time but there is a chance of doing mistakes while buidling the trees.

Shortcut way!

Hello

H - 1
e - 1
l - 2
o - 1



H → 00
e → 01
o → 10
l → 11

Total = 8 bits.
If it was done by ASCII,
total = 5x8 = 40 bits.

For example, if the input sentence is:

Eerie eyes seen near lake.

Each character occupies 8 bits. There are a total of 26 characters. Thus, a total of 8*26 = 208 bits is required. We can compress the string to a smaller size using the Huffman Coding technique.

Huffman coding first creates a tree using the frequencies of the characters and then generates code for each character.

Character	Frequency	Character	Frequency
E	1	s	2
e	8	n	2
r	2	a	2
i	1	l	1
space	4	k	1
y	1	.	1

Basic Algorithm:

1. Scan text to be compressed and tally occurrence of all characters.
2. Sort or prioritize characters based on number of occurrences in text.
3. Build Huffman code tree based on prioritized list.
4. Perform a traversal of tree to determine all code words.
5. Scan text again and create new file using the Huffman codes.

Building a Tree:

1. Create binary tree nodes with character and frequency of each character
2. Place nodes in a priority queue (The lower the occurrence, the higher the priority in the queue)
3. While priority queue contains two or more nodes:
    i. Create new node
    ii. Dequeue node and make it left subtree
    iii. Dequeue next node and make it right subtree
    iv. Frequency of new node = sum of frequency of left and right children
    v. Enqueue new node back into queue

Building a Tree

Scan the original text

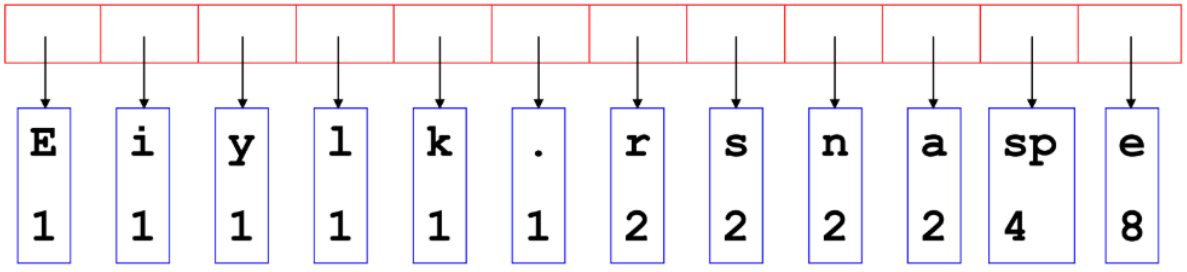
Eerie eyes seen near lake.

- What is the frequency of each character in the text?

Char	Freq.	Char	Freq.	Char	Freq.
E	1	y	1	k	1
e	8	s	2	.	1
r	2	n	2		
i	1	a	2		
space	4	l	1		

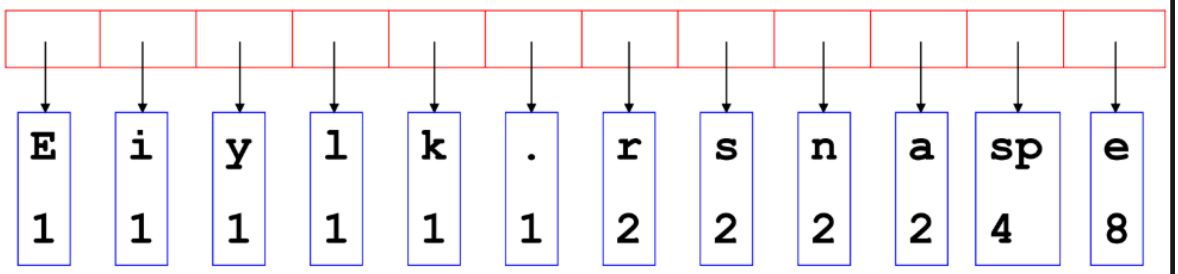
- Create binary tree nodes with character and frequency of each character
- Place nodes in a priority queue
  - The lower the occurrence, the higher the priority in the queue

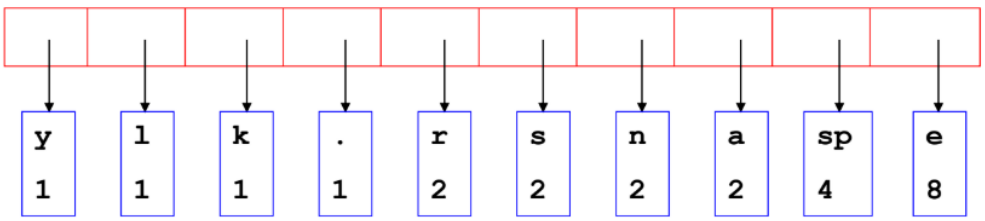
- The queue after inserting all nodes

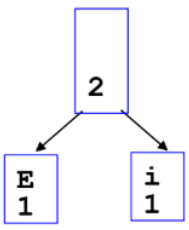


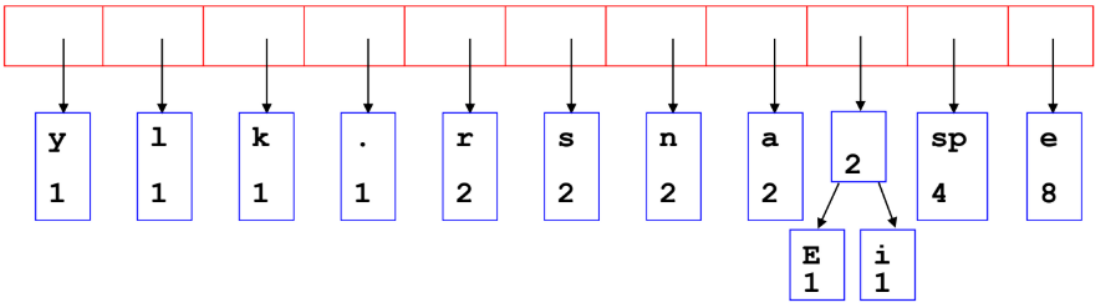
- Null Pointers are not shown

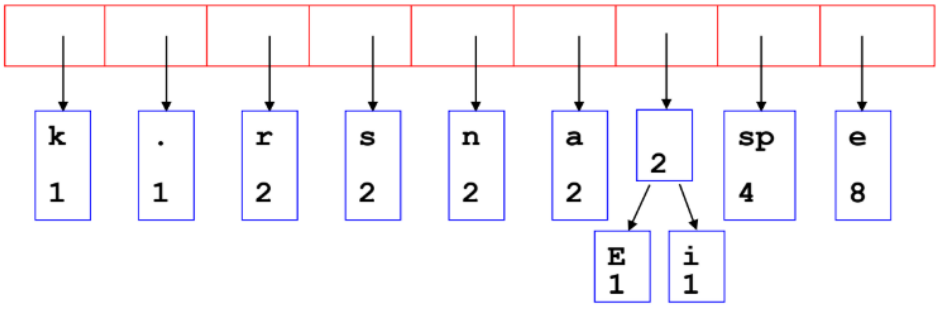
- While priority queue contains two or more nodes
  - Create new node
  - Dequeue node and make it left subtree
  - Dequeue next node and make it right subtree
  - Frequency of new node equals sum of frequency of left and right children
  - Enqueue new node back into queue

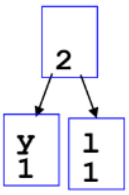


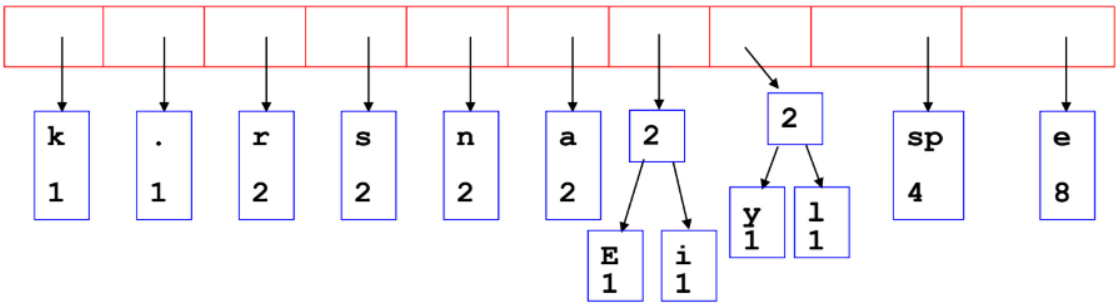


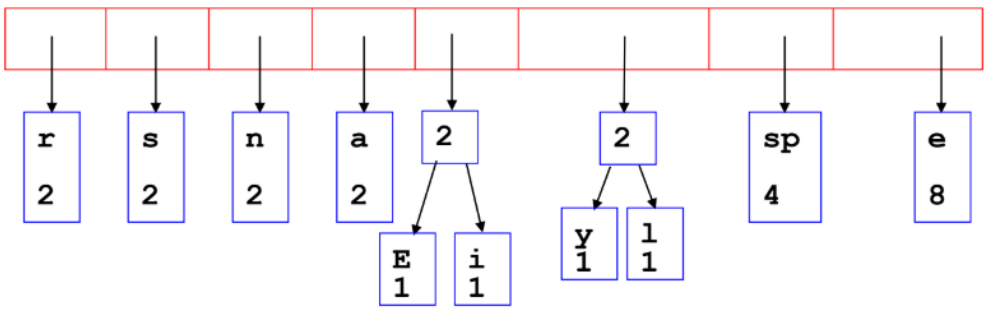


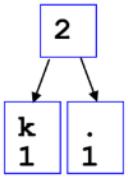

y 1
l 1
k 1
. 1
r 2
s 2
n 2
a 2
2
sp 4
e 8
E 1
i 1

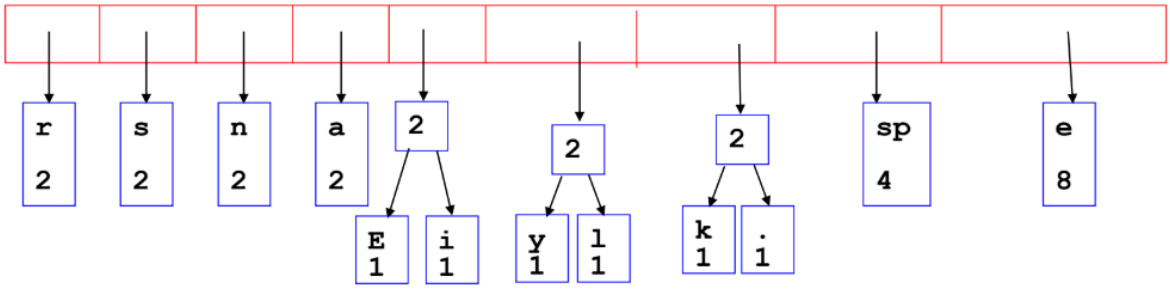

k 1
. 1
r 2
s 2
n 2
a 2
2
sp 4
e 8
E 1
i 1

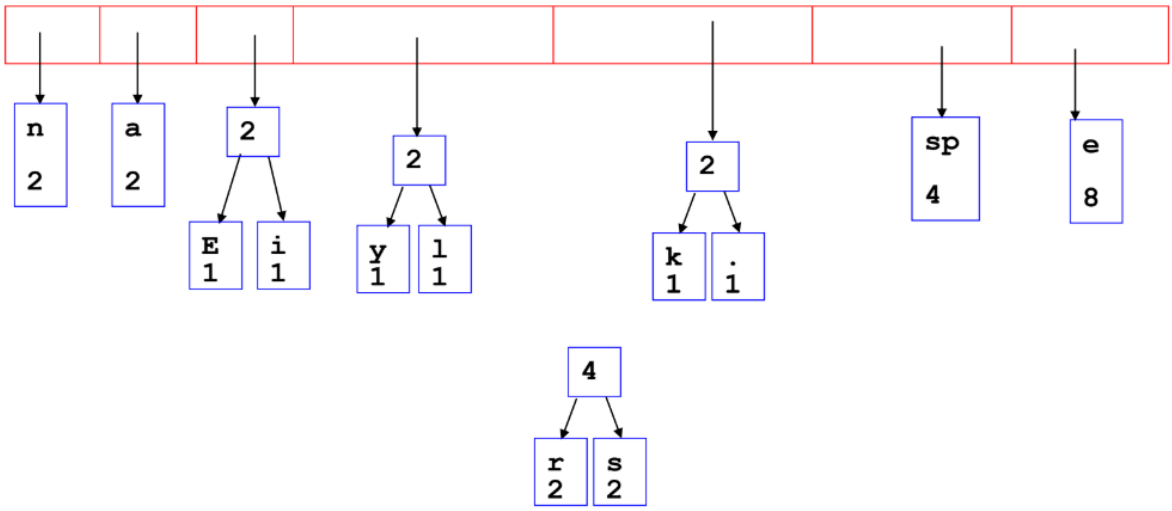

2
y 1
l 1

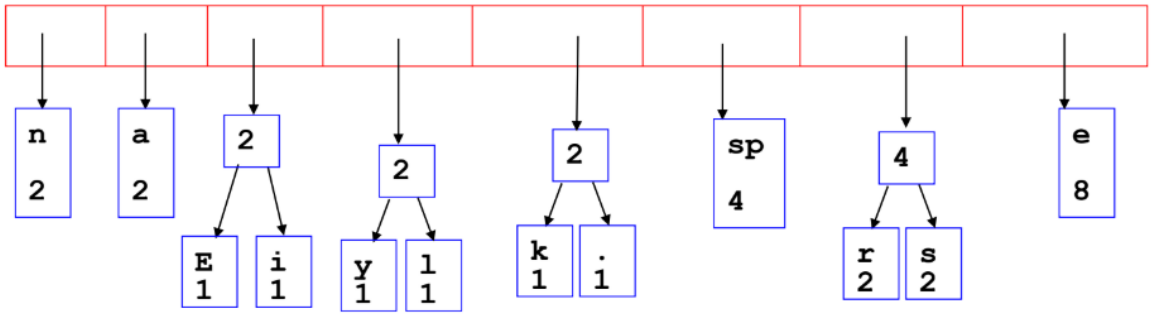

k 1
. 1
r 2
s 2
n 2
a 2
2
2
sp 4
e 8
E 1
i 1
y 1
l 1

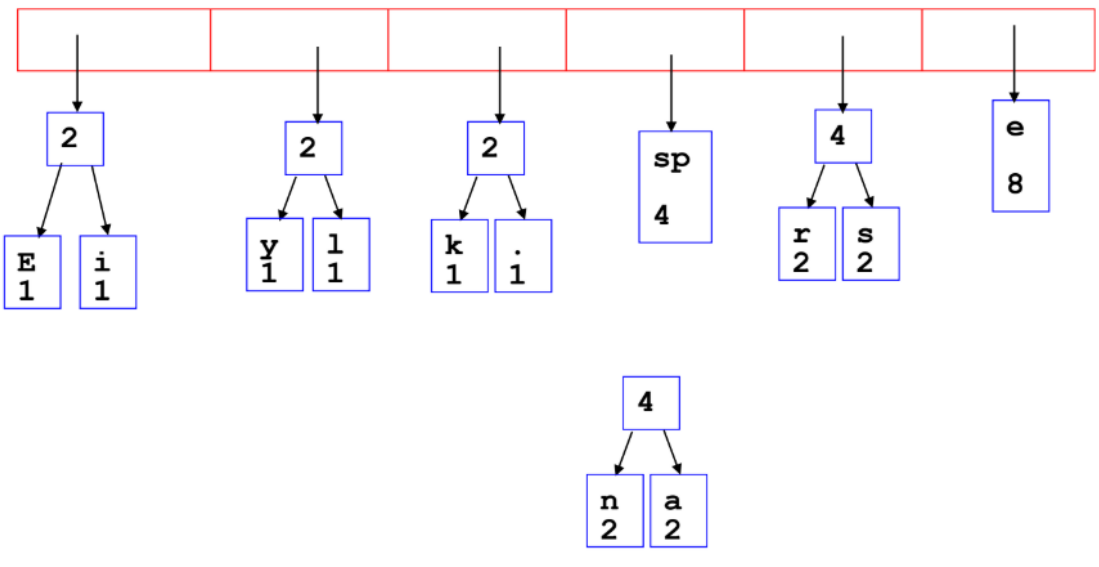

r 2
s 2
n 2
a 2
2
2
sp 4
e 8
E 1
i 1
y 1
l 1

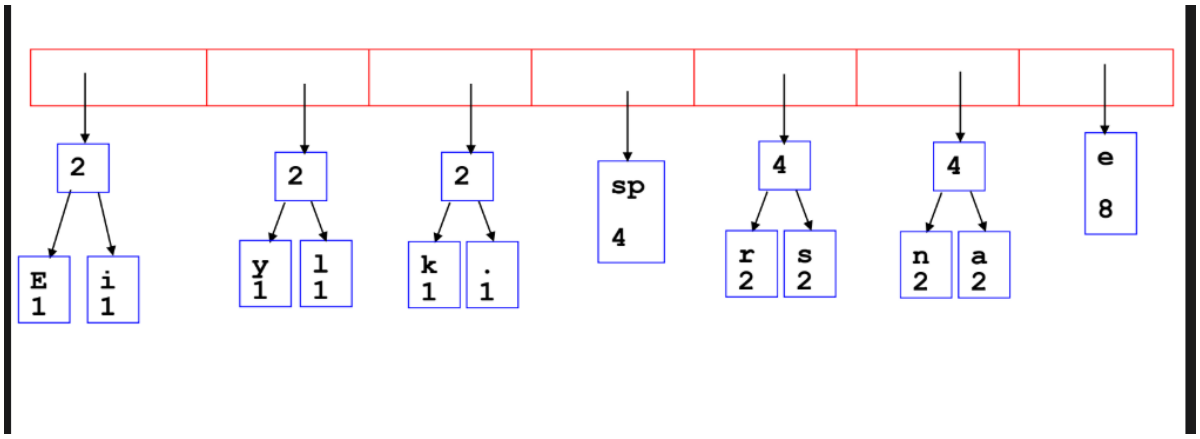

2
k 1
. 1

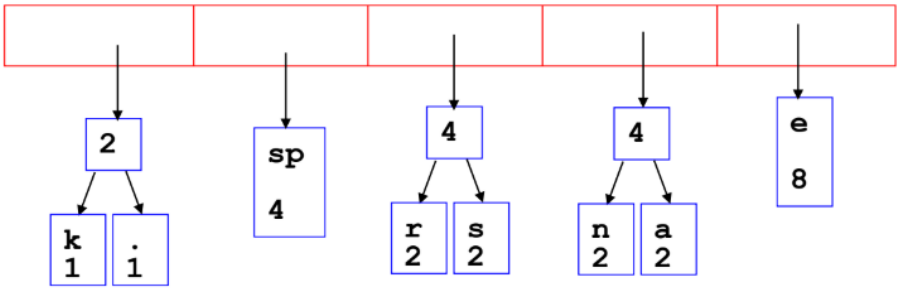

r 2
s 2
n 2
a 2
2
E 1
i 1
2
y 1
l 1
2
k 1
. 1
sp 4
e 8

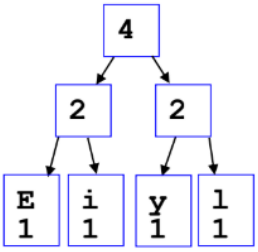

n 2
a 2
2
E 1
i 1
2
y 1
l 1
2
k 1
. 1
sp 4
e 8
4
r 2
s 2

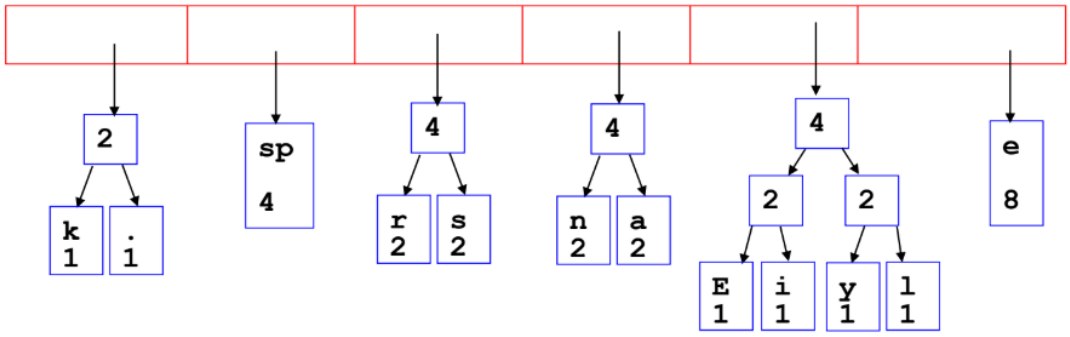

n 2
a 2
2
E 1
i 1
2
y 1
l 1
2
k 1
. 1
sp 4
4
r 2
s 2
e 8

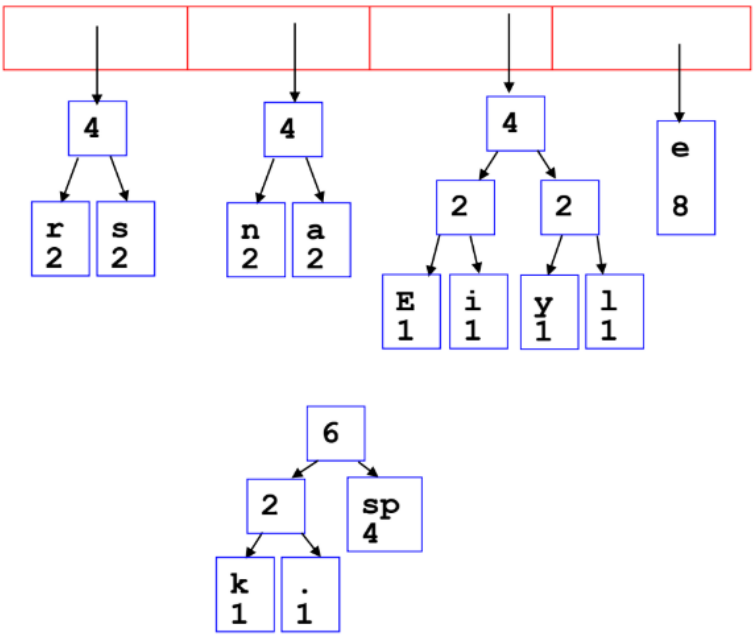

2
E 1
i 1
2
y 1
l 1
2
k 1
. 1
sp 4
4
r 2
s 2
e 8
4
n 2
a 2

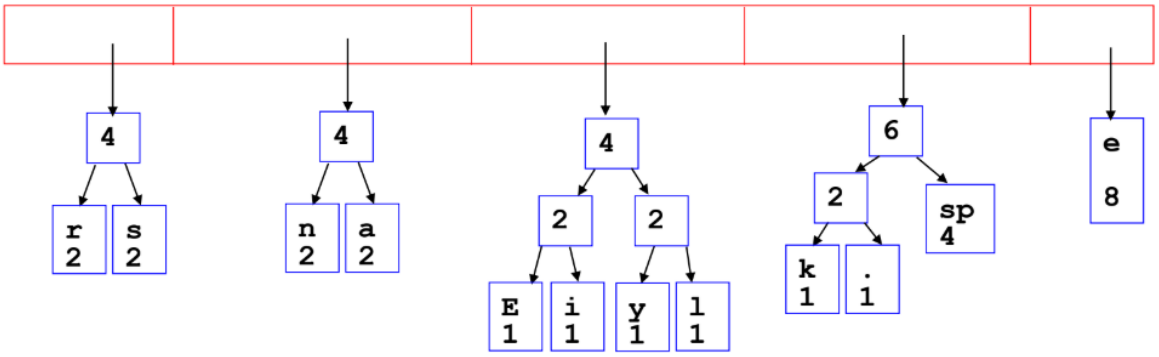

2
E 1
i 1
2
y 1
l 1
2
k 1
. 1
sp 4
4
r 2
s 2
4
n 2
a 2
e 8


2
k 1
. 1
sp 4
4
r 2
s 2
4
n 2
a 2
e 8

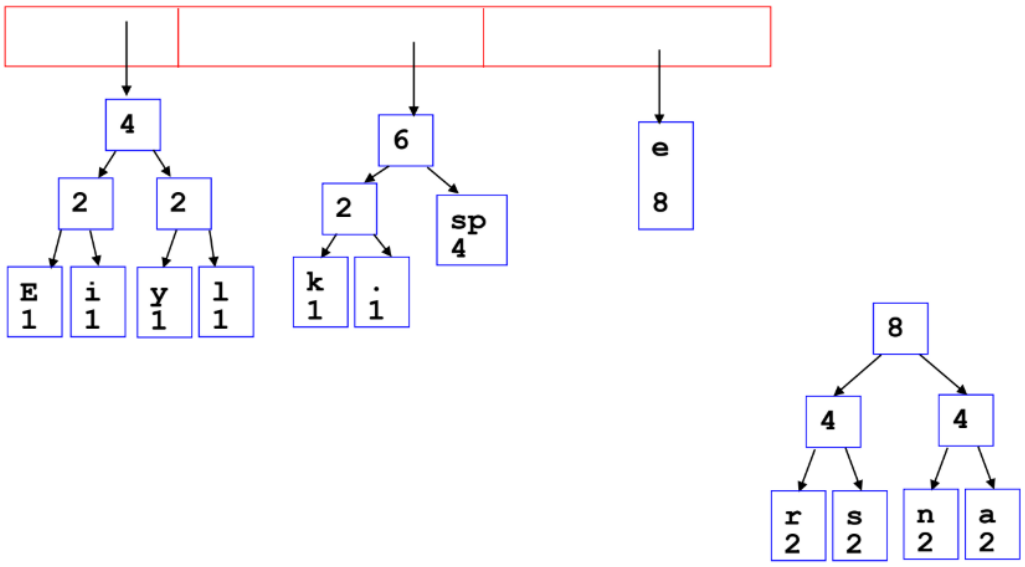

4
2
2
E 1
i 1
y 1
l 1

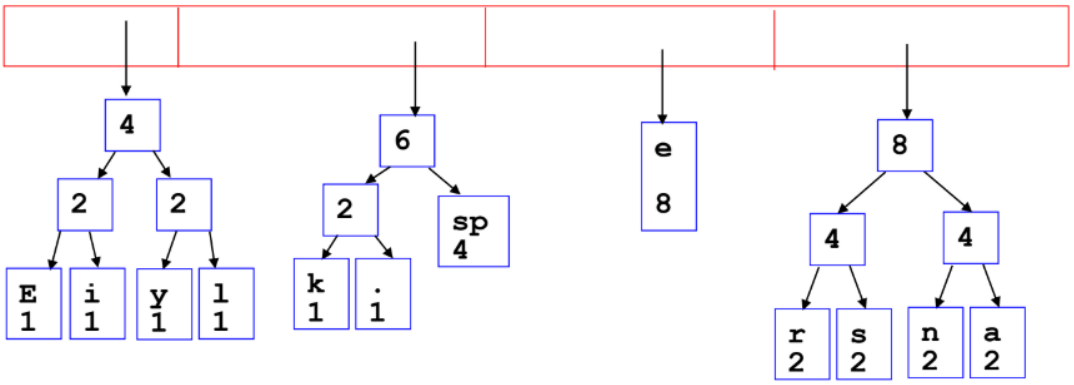

2
k 1
. 1
sp 4
4
r 2
s 2
4
n 2
a 2
4
2
2
E 1
i 1
y 1
l 1
e 8

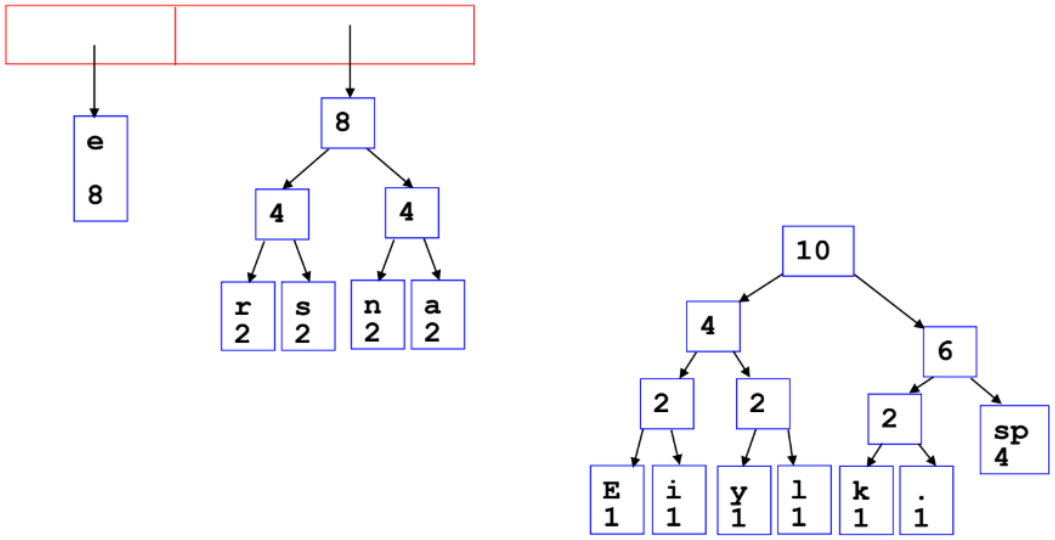


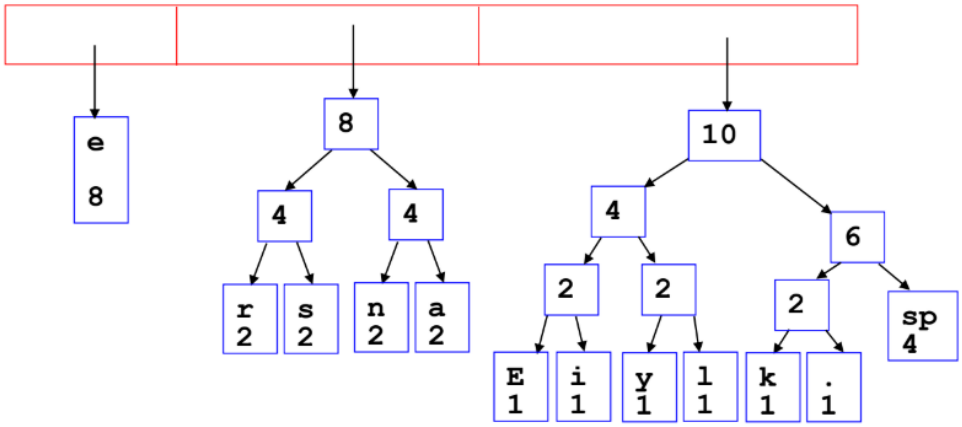


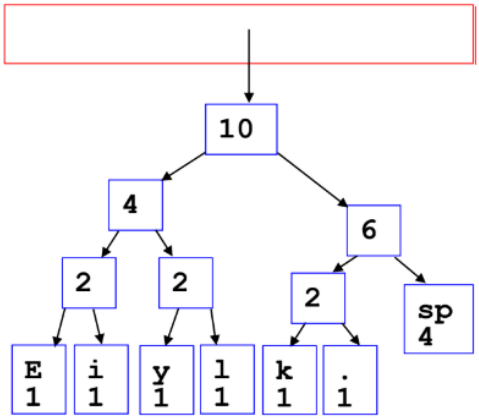
What is happening to the characters with a low number of occurrences?

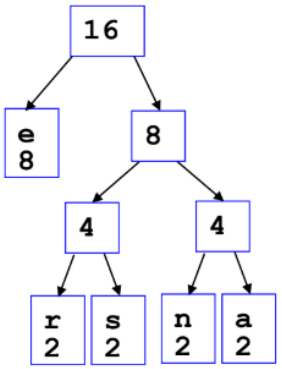


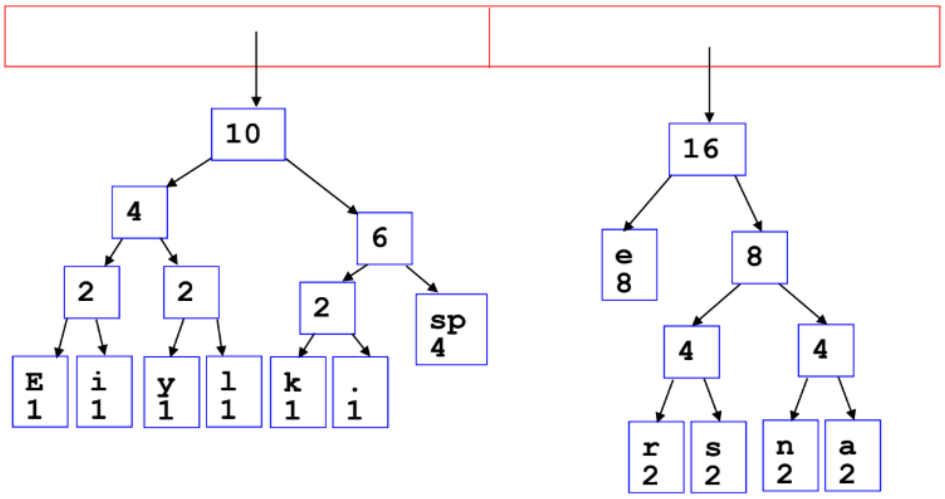

4
2
2
E 1
i 1
y 1
l 1
6
2
sp 4
k 1
. 1
e 8
8
4
4
r 2
s 2
n 2
a 2

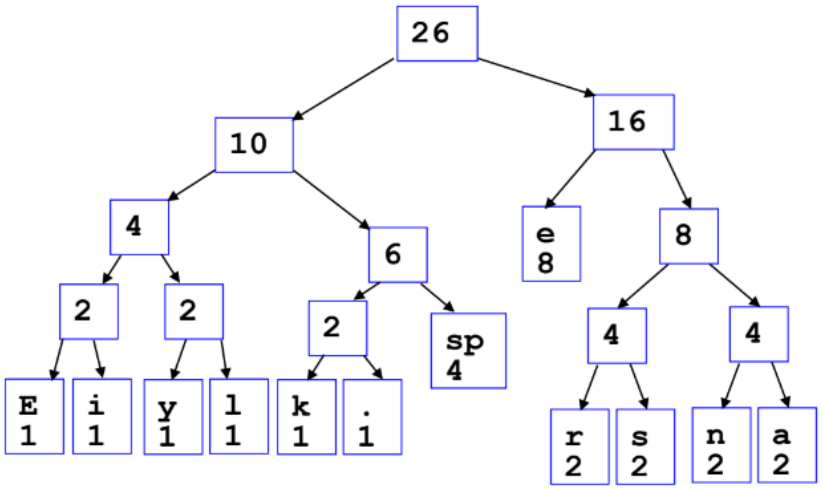

e 8
8
4
4
r 2
s 2
n 2
a 2
10
4
6
2
2
2
sp 4
E 1
i 1
y 1
l 1
k 1
. 1

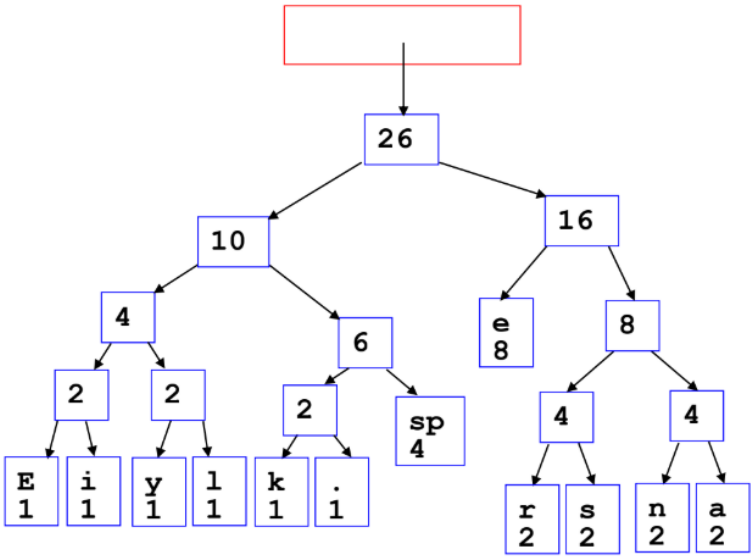

e 8
8
4
4
r 2
s 2
n 2
a 2
10
4
6
2
2
2
sp 4
E 1
i 1
y 1
l 1
k 1
. 1


10
4
6
2
2
2
sp
4
E
1
i
1
y
1
l
1
k
1
.
1


16
e
8
8
4
4
r
2
s
2
n
2
a
2


10
4
6
2
2
2
sp
4
E
1
i
1
y
1
l
1
k
1
.
1
16
e
8
8
4
4
r
2
s
2
n
2
a
2


26
10
16
4
6
e
8
8
2
2
2
sp
4
4
4
E
1
i
1
y
1
l
1
k
1
.
1
r
2
s
2
n
2
a
2

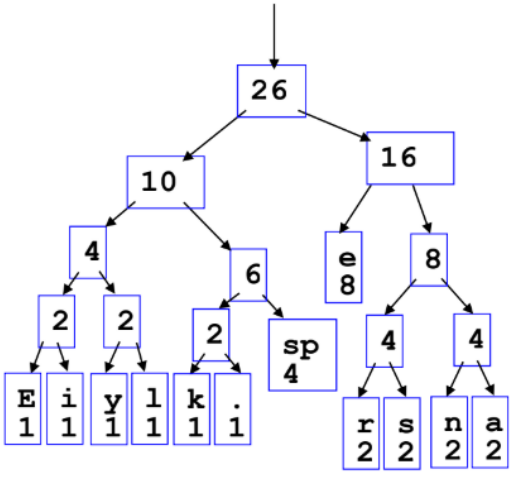


•After enqueueing this node there is only one node left in priority queue.

Dequeue the single node left in the queue.

This tree contains the new code words for each character.

Frequency of root node should equal number of characters in text.

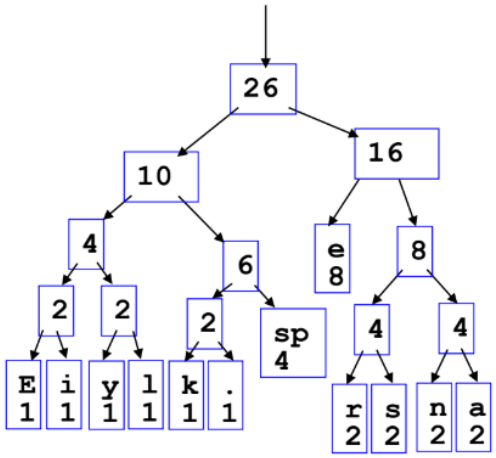


Eerie eyes seen near lake. 26 characters

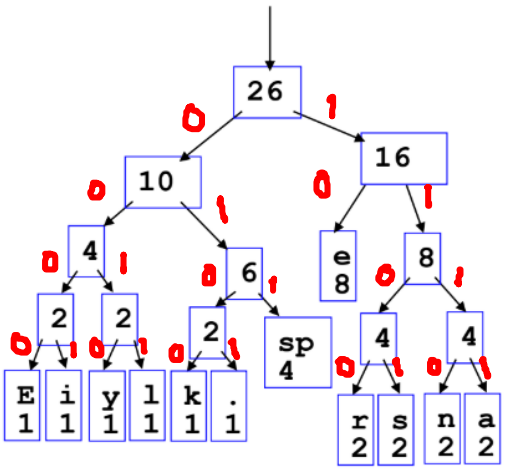
Encoding the File

Traverse Tree for Codes

- Perform a traversal of the tree to obtain new code words
- Going left is a 0 going right is a 1
- code word is only completed when a leaf node is reached



Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111



Q: If asked, the sum of the Only Encoded

Char	Freq.	Char	Freq.	Char	Freq.
E	1	y	1	k	1
e	8	s	2	.	1
r	2	n	2		
i	1	a	2		
space	4	l	1		

Char	Code	
E	0000	→ 4x1 = 4
i	0001	→ 4x1 = 4
y	0010	→ 4x1 = 4
l	0011	→ 4x1 = 4
k	0100	→ 4x1 = 4
.	0101	→ 4x1 = 4
space	011	→ 3x4 = 12
e	10	→ 2x8 = 16
r	1100	→ 4x2 = 8
s	1101	→ 4x2 = 8
n	1110	→ 4x2 = 8
a	1111	→ 4x2 = 8
		Σ = 84

Text: Eerie eyes seen near lake.

Encoded: 00001011000101001110010101101101011011101110111010111110001100111111010010101

84 bits are required to represent instead of 208 bits (ASCII). Hence, there are savings.

Encoding the File

- Rescan text and encode file using new code words

Eerie eyes seen near lake.

```
00001011000001100 11
10001010110110100 11
11101011111100011 00
11111101001001 01
```

- Why is there no need for a separator character?

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

Encoding the File

Results

- Have we made things any better?
- 73 bits to encode the text

```
00001011000001100 11
10001010110110100 11
11101011111100011 00
11111101001001 01
```

- ASCII would take 8 * 26 = 208 bits
- If modified code used 4 bits per character are needed. Total bits 4 * 26 = 104. Savings not as great.

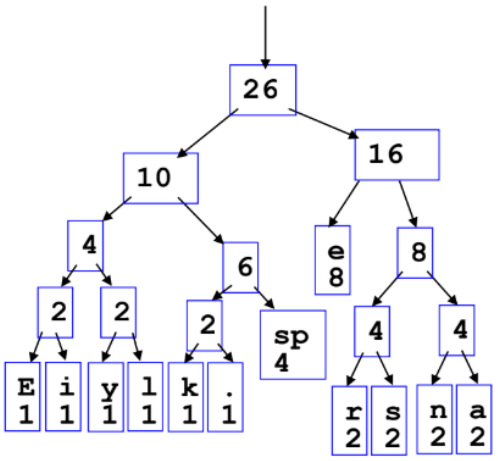
Decoding the File

- How does receiver know what the codes are?
- Tree constructed for each text file.
  - Considers frequency for each file
  - Big hit on compression, especially for smaller files
- Tree predetermined
  - based on statistical analysis of text files or file types

Decoding the File

- Once receiver has tree it scans incoming bit stream
- 0 ⇒ go left
- 1 ⇒ go right

```
101000110111101111
0111111000110101
```



N.B. When we will get a character, will return to the Root Node. Then same procedure again and again.

Time Complexity:

Frequency count (m = input size)	O(m) [Linear Search]
Min-heap creation (n = unique characters)	O(n log n) [Swim n nodes]
Build a Huffman tree	O(n log n) [Delete 2 smallest n-1 times]
Generate codes (DFS)	O(n) [Assigning codes for each leaf nodes]
Total	O(m + n log n)

Summary

- Huffman coding is a technique used to compress files for transmission
- Uses statistical coding
  - more frequently used symbols have shorter code words
- Works well for text and fax transmissions
- An application that uses several data structures